

Probe Reference — Decision Boundary Playground

A complete guide to every probe in `playground/probes/`. For each probe: what it measures, the equation it implements, the intuition, how it's coded, when it works, when it fails, and what our experiments told us.

This is the document to read when:

- A probe disagrees with another and you want to know why.
- You're picking which probe to add to a new sweep.
- A probe gave a surprising number and you want to know if the *number* is wrong or your *expectation* is wrong.

Background — what are we even measuring?

A trained model $f(x)$ partitions input space $\mathbb{R}^d$ into regions, one per class. The **decision boundary** is the set of points where the model is indifferent between two classes. **Brittleness** means: a tiny perturbation in input pushes a point across the boundary, changing the prediction. Equivalently, the boundary is *close* to the data points in input space.

This can be measured in **two different geometric spaces**:

Space	What lives there	"Sharpness" means
Parameter space ($\Theta \subset \mathbb{R}^n$)	θ	$\nabla_{\theta} f(x)$
Input space ($X \subset \mathbb{R}^d$)	The data x	The function $f(x)$ has high slope near data — small input perturbations spike the logits, flipping predictions

These two spaces are *connected* (a sharp loss minimum tends to have large weight norms which give a steep f), but the connection is empirical, not exact. **The Andriushchenko 2023 result is essentially: param-space sharpness and input-space brittleness can disagree.** Probes that measure one don't automatically measure the other.

Each probe lives in exactly one of these spaces. The table below classifies them.

#	Probe	Space	Cost	Status
P1	Margin distribution	Output (logit)	Cheap	✓ working but misleading alone
P2	Input-gradient norm	Input (loss surface)	Cheap	⚠ measures loss-grad not f-grad — saturates
P3	Adversarial ϵ -curve (FGSM/PGD)	Input	Medium	✓ operational gold standard
P4	Boundary thickness	Input	Slow	✗ broken (unbatched, hangs on CIFAR)
P5	Hessian top-k eigenvalues	Parameter	Expensive	✓ parameter-space gold standard
P6	Filter-normalized landscape slice	Parameter	Expensive	⚠ visualization fails at default span
P7	Linear interpolation between checkpoints	Parameter	Medium	✓ working (rarely the right probe though)
P8	Calibration / ECE	Output (probability)	Cheap	✓ working but not a brittleness probe

P1 — Margin distribution

File: `playground/probes/margin.py` → `probe_margin(model, loader, device)` -> dict **Space:** Output / logit space

The equation

For each test sample (x, y) , define the **logit margin**:

$$m(x, y) = f(x)_y - \max_{j \neq y} f(x)_j$$

- $f(x)_y$ is the logit of the true class.
- $\max_{j \neq y} f(x)_j$ is the largest logit among wrong classes.

- $m > 0 \Leftrightarrow$ prediction is correct.
- $|m|$ measures how *confidently* (in logit terms) the model picked its answer.

We compute m for every test point and report the histogram + summary statistics (mean, p10, p50, p90, frac_negative).

Implementation (the trick)

```
logits = model(x)                                # shape (B, C)
correct = logits.gather(1, y.unsqueeze(1)).squeeze(1) # the y-th logit per row
masked = logits.clone()
masked.scatter_(1, y.unsqueeze(1), float("-inf"))    # blank out the y-th col
max_other, _ = masked.max(dim=1)
margins = correct - max_other
```

The `scatter_(-inf)` trick lets us take "max over other classes" in one pass — replacing the true-class entry with $-\infty$ ensures `.max()` ignores it.

Intuition

Margin is a **confidence** measure. A model that puts logit 10 on the true class and 0 on others has margin 10 — very confident. A model that puts 0.6 on true and 0.4 on others has margin 0.2 — barely sure.

When it gives clean signal

- Comparing **calibration** between similar models.
- Detecting **overconfidence** (large positive margin without correspondingly low loss).

When it fails / is misleading

P1 is NOT a reliable brittleness probe. Here's why:

The logit margin lives in *output space*, not input space. A model can have a huge logit margin while sitting arbitrarily close to the decision boundary in *input* space, if the function f is very steep. Two networks:

- Net A: $f(x) = 100x$, sitting at $x=0.07$. Margin in logit space = 7. But the boundary ($f=0$) is just 0.07 away in input space.
- Net B: $f(x) = 3x$, sitting at $x=1.0$. Margin = 3. Boundary is 1.0 away.

Net B has a *smaller* logit margin but is *more* robust. P1 alone can't tell you which is the case — you need P3 (adversarial) or knowledge of weight norms.

Empirical results

From Exp 1 (Section §1 of `docs/runlog/2026-05-14.md`):

batch_size	margin_mean	robust@8/255
32	4.99	1.4%
8192	0.60	17.7%

The brittle small-batch model has *higher* margin but *worse* robustness. P1 disagrees with P3. This was our smoking gun for "margin $\neq$ brittleness."

From Exp 2:

batch_size	margin_mean	robust@8/255
256	16.8	8.4%
5000	5.9	0.9%

Here P1 and P3 *agree* — small batch has higher margin AND better robustness. But this agreement is coincidental: it happens because both models are well-converged and weight norms grew along with margin. Don't trust agreement without checking weight norms.

P2 — Input-gradient norm

File: `playground/probes/input_grad.py` → `probe_input_grad(model, loader, device)` → dict **Space:** Input space (loss surface)

The equation

For each test sample, compute the L2 norm of the loss gradient with respect to the input:

$$g(x, y) = \|\nabla_x \ell(f(x), y)\|_2$$

Where ℓ is cross-entropy. Higher g means the loss surface is steep around x — a small input nudge spikes the loss.

Implementation

```
x.requires_grad_(True)
loss = cross_entropy(model(x), y)
grad = torch.autograd.grad(loss, x)[0]
norms = grad.flatten(start_dim=1).norm(p=2, dim=1)
```

We aggregate over the full test set and report mean, median, p90.

The intuition (and why it's subtle)

A naive reading: high $\|\nabla_x \ell\| \Rightarrow$ small input perturbation $\Rightarrow$ big loss $\Rightarrow$ brittle. **This is partly true but masked by softmax saturation.**

When a model is very confident on x (margin large, softmax $\approx$ one-hot):

- The cross-entropy loss is near zero: $\ell \approx 0$.
- $\partial \ell / \partial (\text{logit}_y) = \text{softmax}_y - 1 \approx 0$.
- So $\nabla_x \ell = (\partial \ell / \partial \text{logit}) \cdot (\partial \text{logit} / \partial x) \approx 0 \cdot (\text{large}) = \text{small}$.

The *loss* gradient vanishes even when the *logit* gradient is large. P2 measures the former, not the latter.

When it gives clean signal

- For *poorly* fit or *uncalibrated* models, where softmax hasn't saturated.
- For comparing models at the *same level* of confidence.

When it fails

- **For well-trained, confident models** — softmax saturation drives $\nabla_x \ell$ toward zero regardless of how steep f is. Both brittle and robust well-trained models can have similar low $\|\nabla_x \ell\|$.

Empirical results

From Exp 1 brittleness demo (rigorous test): brittle model had `mean_grad_norm = 0.108`, smooth model had `0.132` — the smoother model has *higher* gradient norm! This is the softmax-saturation paradox: brittle was so over-confident its gradient was suppressed.

Recommended fix (Side fix E in the runlog backlog)

Replace $\nabla_x \ell$ with $\nabla_x m(x, y)$ — the gradient of the **logit margin**, not the loss. The margin gradient doesn't saturate. Equation:

$$g'(x, y) = \|\nabla_x (f(x)_y - \max_{j \neq y} f(x)_j)\|_2$$

This would make P2 a real input-space sharpness probe instead of a loss-curvature probe.

P3 — Adversarial ϵ -curve (FGSM and PGD-k)

File: `playground/probes/adversarial.py` $\rightarrow$ `probe_adversarial(model, loader, device, epsilons, attack, pgd_steps, pgd_alpha) -> dict`
Space: Input space — **this is the operational gold standard**

The equation

Given attack budget ϵ , find a perturbation δ with $\|\delta\|_\infty \leq \epsilon$ that maximizes loss:

$$\delta^* = \operatorname{argmax}_{\{\|\delta\|_\infty \leq \epsilon\}} \ell(f(x + \delta), y)$$

Then evaluate the model on $x + \delta^*$. Report accuracy as a function of ϵ .

FGSM (Fast Gradient Sign Method, Goodfellow 2015) — single step

$$\delta_{\text{FGSM}} = \epsilon \cdot \operatorname{sign}(\nabla_x \ell(f(x), y))$$

One gradient step in the direction that most increases loss. Cheap but only an approximate maximizer.

PGD-k (Projected Gradient Descent, Madry 2018) — iterative

```
δ_0 = uniform(-ε, ε)
for i in range(k):
    δ_{i+1} = clip( δ_i + α · sign(∇_x ℓ(x+δ_i, y)) , [-ε, ε] )
```

Many small steps with re-projection onto the ϵ -ball. Much stronger attacker than FGSM. We use $k=20$, $\alpha=\epsilon/4$ as standard.

Implementation note

Both `_fgsm` and `_pgd` operate **in the input space the model sees** — i.e., post-normalization. This means ϵ is measured in normalized pixel units, not $[0, 255]$ pixel units. For CIFAR-10 normalized with $\sigma \approx 0.25$, a perturbation budget $\epsilon = 8/255 \approx 0.031$ in raw pixels corresponds to $\epsilon \approx 0.124$ in normalized input space. Be careful when comparing to papers that report ϵ in raw pixels.

Why this is the operational gold standard

- It directly answers the question "would a small input perturbation flip the prediction?"
- It's the threat model deployed adversarial defenses target.
- It doesn't require any assumptions about the model's internal structure.

When it gives clean signal

Always, as long as you:

- Pick reasonable ϵ values for the dataset normalization (we use $\{0, 1, 2, 4, 8, 16\} / 255$ for CIFAR).
- Use PGD-20 or stronger; FGSM is a weaker attack and overestimates robustness.
- **Normalize by clean accuracy when comparing across models** with different clean accuracies — use the **fragility ratio** $(\text{clean} - \text{robust}) / \text{clean}$. Our Exp 1 showed why: bs=8192's "high robust acc" was just because clean acc was already low.

When it fails

- **At extreme batch sizes / underfit models**, where the model is half-trained. PGD reports "robustness" that's really untrained-randomness.
- **With incorrect ϵ scaling** (raw vs normalized pixel units).

Empirical results

Exp 2 (Keskar faithful):

bs	clean	robust@8/255	fragility
256	0.844	0.084	0.901
5000	0.796	0.009	0.989

Welch's $t = 17.4$, $p = 0.0008$. Even with 3 seeds, this gap is rock-solid.

P4 — Boundary thickness (Yang 2020)

File: `playground/probes/boundary_thickness.py` → `probe_boundary_thickness(model, loader, device, n_directions, step, max_steps)` → dict
Space: Input space — **currently broken at CIFAR scale**

The equation

For each test sample x with prediction y_{pred} , draw n random unit-norm directions d_i and find the smallest step size where the prediction flips:

$$t^*_i(x) = \min \{ t > 0 : \arg\max f(x + t \cdot d_i) \neq y_{\text{pred}} \}$$

The **boundary thickness** for x is the mean over directions:

$$\text{thickness}(x) = (1/n) \sum_i \min(t^*_i, \text{max_steps} \cdot \text{step})$$

If no flip is found within `max_steps`, that direction contributes the maximum (capped distance).

The intuition

This is the *geometric* version of "distance to the boundary" — average over random directions. Larger thickness $\Leftrightarrow$ data sits farther from the decision surface in input space $\Leftrightarrow$ more robust.

Why it's slow (and currently broken)

The naïve implementation iterates one sample at a time, one direction at a time, one step at a time. For CIFAR-10 with the default settings:

```
operations = N_test × n_directions × max_steps × forward_pass_cost
            = 10,000 × 16 × 64 × (PreActResNet-20 fwd)
            ≈ 10.2 million forward passes
```

At ~5 ms per single-sample forward on an A10, that's ~14 hours per call. The probe hung in Exp 1.

For MNIST + MLP-2 the same probe finished in ~30 seconds because both factors are smaller.

How to fix (Side fix A)

Batch the perturbation over the entire test set per direction:

```
for direction_idx in range(n_directions):
    direction = ... # one random direction, shape (1, *input_shape)
    for step_idx in range(max_steps):
        x_perturbed = x_batch + step_idx * step * direction
        new_preds = model(x_perturbed).argmax(1)
        # for each sample, record the first step where prediction != original
```

This turns ~10M sequential forward passes into ~64 batched forwards per direction × 16 directions = ~1000 batched calls. Should take <5 min on A10.

Status

Currently disabled in the executor. `playground/sweep/executor.py:60` uses a "deep-but-safe" probe list `margin,grad,adv,calibration,hessian,landscape` that excludes P4 until Side fix A lands.

Empirical results from Exp 1 brittleness demo (MNIST, where it ran)

condition	thickness
brittle	0.4999 ± 0.0000
smooth	0.4999 ± 0.0001

Both saturated at the max because we used `step=0.05`, `max_steps=10` (cap = 0.5), and every sample took more than 0.5 of input-space perturbation to flip on MNIST. Probe gave no signal — needed `step=0.5`, `max_steps=20` to span enough range. **Configuration matters as much as code.**

P5 — Hessian top-k eigenvalues (Pearlmutter 1994 + Lanczos)

File: `playground/probes/hessian.py` → `probe_hessian_top_eigenvalues(model, loader, device, k, max_batches)` → dict **Space:** Parameter space — **parameter-space gold standard**

The equation

The loss Hessian H at the trained parameters θ^* is the matrix of second partials:

$$H_{\{ij\}} = \partial^2 L(\theta) / \partial \theta_i \partial \theta_j \quad \text{evaluated at } \theta = \theta^*$$

For a network with N parameters, H is $N \times N$ — far too large to store (N is in the hundreds of thousands for SmallCNN, millions for ResNets). We can't materialize H , but we can compute H times any vector v (a **Hessian-vector product** or HVP) via:

$$H \cdot v = \nabla_{\theta} ((\nabla_{\theta} L) \cdot v)$$

This requires two backward passes through the network — cheap. Pearlmutter 1994 is the canonical reference.

We use Hv to compute the **top-k eigenvalues** of H via Lanczos iteration (via `scipy.sparse.linalg.eigsh` on a custom `LinearOperator` that does HVP for us).

Implementation (the magic)

```
def hvp(v_np):
    v = torch.from_numpy(v_np).to(device)
    # chunk v back into per-parameter shapes
    chunks = split_v_across_params(v, params)
    total = torch.zeros(n, device=device)
    for x, y in cached_batches:
        loss = F.cross_entropy(model(x), y)
        grads = torch.autograd.grad(loss, params, create_graph=True)  # 1st backward
        dot = sum((g * c).sum() for g, c in zip(grads, chunks))
        hvp_parts = torch.autograd.grad(dot, params)  # 2nd backward
        total += flatten(hvp_parts).detach()
    return (total / len(cached_batches)).cpu().numpy()

linop = LinearOperator((n, n), matvec=hvp)
eigvals = eigsh(linop, k=5, which="LA")
```

- `create_graph=True` on the first `.grad()` keeps the autograd graph so we can differentiate again.
- We average over `max_batches=8` cached batches for a stable HVP estimate.
- Lanczos converges quickly for the top eigenvalues (a few hundred matvecs typically suffice).

Why this is the parameter-space gold standard

- It measures **actual curvature** of the loss at the trained weights — what Keskar 2017 and the entire "sharp vs flat minima" literature is about.
- $\lambda_{\max}$ is the most reliable single-number sharpness summary.
- Lanczos via HVP is the standard scientific computing technique for this — proven, fast, exact (up to Lanczos iteration tolerance).

When it gives clean signal

- For **converged models** (loss gradient at θ^* is small).
- When comparing models at **similar loss levels** (sharpness changes meaning if you're comparing converged vs underfit).

When it fails

- **For underfit models:** if the model hasn't converged, you're computing curvature at a non-minimum. The eigenvalues represent the local quadratic approximation around an arbitrary point on the loss surface, not the minimum's basin shape.

This is exactly what bit us in Exp 1 — bs=8192 with 20 epochs hadn't converged, so the Hessian eigenvalues (which we never actually collected, but would have) would have been hard to interpret.

Empirical results

Exp 2 (Keskar faithful):

bs	$\lambda_{\max}$	Top-5 eigenvalues
256	103	[103, 89, 73, 66, 55]
5000	9 773	[9773, 6187, 5053, 4651, 3422]

95× ratio in $\lambda_{\max}$. Every top eigenvalue is ~50–100× larger at bs=5000. This is the cleanest single piece of evidence for the sharp-vs-flat story we've collected.

P6 — Filter-normalized 2D loss landscape (Li 2018)

File: playground/probes/landscape.py → `probe_loss_landscape_slice(model, loader, device, grid_size, span, max_batches, seed) -> dict`
Space: Parameter space (visualization, not measurement)

The equation

Generate two random parameter-space directions d_1, d_2 , then evaluate the loss on a 2D grid:

$$L_{\text{grid}}[i, j] = L(\theta^* + \alpha_i \cdot d_1 + \beta_j \cdot d_2)$$

with $\alpha_i, \beta_j \in [-\text{span}, +\text{span}]$ sampled at `grid_size` points each.

The filter normalization (the special sauce)

A naïve random direction in parameter space gets dominated by whichever parameter tensor has the largest norm. Li et al. 2018's fix: per-filter-rescale each direction so the *direction's filter-norms match the weights' filter-norms*:

For a 4D conv weight tensor W of shape (C_{out}, C_{in}, k, k) , and a random direction tensor d of the same shape:

```
d_filter_norm = ||d[c, :, :, :]|_F (per output channel)
W_filter_norm = ||W[c, :, :, :]|_F
d_normalized[c, :, :, :] = d[c, :, :, :] * (W_filter_norm / d_filter_norm)
```

For 2D Linear weights: same idea per row. For 1D bias/norm parameters: zero out the direction (no meaningful filter).

This makes the visualization *scale-invariant* to weight magnitudes — the stated goal is to make landscape plots comparable across different architectures.

Why this is a visualization, not a measurement

Two random directions in a 100K-dimensional parameter space have essentially zero probability of aligning with the top Hessian eigenvectors. The landscape plot shows the **average** curvature in two random slice planes — not the **worst-case** curvature that P5 reports.

Status

Working, but **visualization fails at the default span**. The contour plots in Exp 2 looked nearly identical for $bs=256$ ($\lambda_{max}=103$) and $bs=5000$ ($\lambda_{max}=9773$), despite a 95× sharpness ratio. **See the deep-dive section below.**

P7 — Linear interpolation between two checkpoints

File: `playground/probes/interp.py` → `probe_linear_interp(model_a, model_b, loader, device, n_points, max_batches)` → dict **Space:** Parameter space

The equation

Given two trained models with parameters θ_A and θ_B , evaluate the loss along the line connecting them:

```

$$\theta(\alpha) = (1 - \alpha) \cdot \theta_A + \alpha \cdot \theta_B$$

$$L\_interp[i] = L(\theta(\alpha_i)) \quad \text{for } \alpha_i \in [0, 1]$$

```

Report (alphas, losses).

Intuition

- Two models in the *same minimum's basin* → $L(\theta(\alpha))$ is monotonically smooth, a small loss bump as α moves away from either endpoint.
- Two models in *different basins* → $L(\theta(\alpha))$ shows a *barrier* — a hump in the middle where you walk between basins through high-loss territory.

This is the empirical test for **mode connectivity** (Garipov 2018 and follow-ups).

Implementation note

The function does a per-key linear interpolation of the state dicts, casts back to original dtype, loads into a temp model, evaluates. We only sample `max_batches=4` batches by default for speed.

When it gives clean signal

- Comparing two models from the *same hyperparameter setup, different seeds* → typically smooth (different basins are still nearby in parameter space).
- Comparing models with *different architectures* or *very different training* → typically shows barriers.

When it's the wrong tool

- For comparing brittleness specifically — it tells you about parameter-space connectivity, not boundary geometry in input space.

Status

Working. Underused in our experiments — we haven't run it yet on the Keskar pairs. Worth a quick run to see if $bs=256$ and $bs=5000$ are in connected basins or not.

P8 — Calibration / Expected Calibration Error (Guo 2017)

File: `playground/probes/calibration.py` → `probe_calibration(model, loader, device, n_bins)` → dict **Space:** Probability (output) space

The equation

Bin predictions by confidence. In each bin b , compute:

```
avg_confidence_b = mean( max softmax probability  for samples in bin b )
avg_accuracy_b   = mean( prediction is correct    for samples in bin b )
```

The **Expected Calibration Error** is the size-weighted gap:

$$\text{ECE} = \sum_b (n_b / N) \cdot | \text{avg_confidence}_b - \text{avg_accuracy}_b |$$

A perfectly calibrated model has $\text{avg_confidence}_b == \text{avg_accuracy}_b$ for every bin $\rightarrow \text{ECE} = 0$.

Implementation

```
probs = torch.softmax(model(x), dim=1)
conf, pred = probs.max(dim=1)
# digitize confidences into bins, compute mean accuracy + mean confidence per bin
```

We report ECE plus per-bin stats for plotting reliability diagrams.

When it gives clean signal

- Comparing models for **production deployment** — well-calibrated confidence is critical when downstream logic uses the confidence as a numeric input.

When it's not a brittleness probe

ECE is about **probability calibration**, not boundary geometry. A model can be perfectly calibrated and still brittle (and vice versa).

Specifically, **label smoothing breaks calibration** by intent — it caps the maximum softmax value, making confidence systematically *under*confident. In our Exp 1 brittleness demo:

condition	ECE	robust@ $\epsilon=0.2$
brittle	0.006	43.2%
smooth	0.109	58.2%

The brittle model had better calibration but worse robustness. ECE and brittleness anticorrelated.

Deep dive — why the landscape probe (P6) didn't show sharp minima

This was the single most counter-intuitive result in Exp 2: numerically the Hessian $\lambda_{\max}$ ratio was **95×**, but the two filter-normalized contour plots looked nearly identical. There are **five compounding reasons**, only the first is really avoidable without code changes.

Setup recap

After Exp 2, we had:

Cell	$\lambda_{\max}$	Basin width*
bs=256 (flat)	103	~0.10
bs=5000 (sharp)	9 773	~0.01

*Basin width estimate: for a quadratic minimum, the loss climbs as $L \approx \frac{1}{2} \lambda t^2$ along the top eigenvector. The width where loss has doubled from $L_{\min}$ is roughly $1/\sqrt{\lambda}$.

We plotted $\alpha, \beta \in [-1.0, +1.0]$ with `grid_size=51`.

Reason 1 — Span was 100× too wide (the dominant cause)

Look at the basin widths:

```
bs=5000 basin width  ≈ 1/√9773  ≈ 0.0101
bs=256  basin width  ≈ 1/√103   ≈ 0.0985
```

Our grid sampled $\alpha \in [-1, +1]$ with 51 points $\rightarrow$ spacing 0.04. For bs=5000, **the entire basin fits inside a single grid cell** near $\alpha=\beta=0$. Every other grid point is sampling terrain far outside the basin where the loss is dominated by "fully saturated cross-entropy on a random model" — the asymptotic ceiling.

Effectively, both contour plots are showing the *same far-from-basin loss surface*. The basin structure that distinguishes them is invisible at our sampling resolution.

Fix: auto-tune the span based on the Hessian eigenvalue (which P5 already computes):

```
span = c / math.sqrt(hessian_lambda_max)    # c  $\approx$  3-5 (basin widths)
# bs=256:  span  $\approx$  0.30  (vs default 1.0)
# bs=5000: span  $\approx$  0.03  (vs default 1.0)
```

This makes each model's visualization centered on *its own basin*. Doesn't make them directly comparable in absolute coordinates, but does make each one visually informative.

Reason 2 — Filter normalization partially cancels the effect

Li et al. 2018 introduced filter normalization to make landscape plots *comparable across architectures*. Mechanism:

$$\| \text{direction_per_filter} \| = \| \text{weight_per_filter} \|$$

The bs=5000 model has *larger weight norms* (sharp basins are reached via aggressive optimization updates that grow weights). After filter normalization, that gives bs=5000 *larger perturbations per unit α* — partially cancelling its sharper basin.

This is great for cross-architecture comparison and bad for same-architecture-different-recipe comparison. Andriushchenko 2023 specifically flags this confound.

Fix: for same-architecture comparisons, use directions normalized by the *whole-network* weight norm rather than per-filter. Or skip normalization entirely if you don't care about cross-arch comparability.

Reason 3 — Random directions almost never hit the top eigenvectors

In a parameter space of dimension $N \approx 100K$, two random Gaussian directions span a 2D plane. The probability that this plane contains the top Hessian eigenvector is essentially zero. The contour plot is showing **average curvature** along its sample plane:

$$\begin{aligned} \text{average_curvature} &\approx \text{tr}(H) / N \\ \text{top_curvature} &= \lambda_{\text{max}} \end{aligned}$$

For our bs=5000 model, $\lambda_{\text{max}} = 9773$ but the *average* eigenvalue is much smaller (most of the N eigenvalues are tiny). So the random slice shows mild curvature even when the worst direction is steep.

Fix: explicitly take $d1$ = top eigenvector from P5 (we compute it for free), then random $d2$ perpendicular to $d1$. This visualizes the *worst-case* slice — the dimension that drives all the brittleness — instead of a noisy average.

Reason 4 — Cross-entropy saturates at log(C) far from minimum

At points 1.0 units away from a converged minimum, both models behave nearly randomly (predicting uniformly across classes). Cross-entropy in that regime is $-\log(1/C) = \log(C)$. For CIFAR-10 with $C=10$, that's $\log 10 \approx 2.30$.

Both contour plots show this ceiling at their edges, making the loss range look similar across models. The *interior* basin differences get crushed into the bottom 5% of the color range.

Fix: plot $\log(L - L_{\text{min}})$ rather than L directly, or fix vmin/vmax per plot to expose the small-perturbation regime.

Reason 5 — Linear contour levels visually squash the sharp basin

`matplotlib.contourf(..., levels=25)` divides the loss range into 25 equal-spaced bands. The bs=5000 plot's dynamic range (0.0 to 2.3, say) gets divided into bands of width 0.1. The basin itself ($\text{loss} < 0.5$) gets just ~ 5 levels. The structure is there in the underlying grid but invisible to the eye.

Fix: `matplotlib.colors.LogNorm()` for the color mapping, or `LogLocator` for contour level placement.

Summary table — fixes ranked by impact

Fix	Where	Code complexity	Visual impact
Auto-tune span from P5's λ_{max}	<code>landscape.py</code>	+5 lines	★★★ (basin actually fits in frame)
Project along top eigenvector	<code>landscape.py</code>	+20 lines	★★ (visualizes worst-case slice)
Log-scale contours	analysis scripts	+1 line	★★ (exposes basin floor)
Drop filter-normalization for same-arch	<code>landscape.py</code>	+5 lines	★ (reveals scale-induced sharpness)

What to use when — decision matrix

Question	Best probe(s)	Why
Is this model brittle in deployment?	P3 (adversarial)	Operational definition; directly answers
Is this minimum sharp in parameter space?	P5 (Hessian)	Direct measurement of curvature
How fragile is the boundary geometrically?	P3 + P4 (when P4 is fixed)	P3 measures worst-case, P4 measures average
Is the model overconfident?	P1 (margin) + P8 (ECE)	Margin → logit-space confidence; ECE → calibration
Are two training runs in the same basin?	P7 (linear interp)	Detects mode connectivity
Visualize the basin shape	P6 (landscape), <i>but tune span first</i>	Pretty pictures, prone to misinterpretation

Cross-probe agreement notes

When two probes *agree*: you have a high-confidence finding (Exp 2: P3 and P5 both said bs=5000 is sharper/more brittle).

When two probes *disagree*: this is *research signal*, not noise. Examples:

- **P1 (margin) vs P3 (adversarial)** disagree → the model is overconfident-but-brittle, or underconfident-but-robust. Usually points to label smoothing, mixup, or saturation.
- **P5 (Hessian) vs P3 (adversarial)** disagree → parameter-space sharpness and input-space brittleness are decoupled. This is Andriushchenko 2023's empirical regime. Worth investigating.
- **P2 (input-grad) vs P3 (adversarial)** disagree → softmax saturation is suppressing your loss gradient. P2's measurement is unreliable for the model state. Switch to margin-gradient.

Status summary

Probe	Status	Action needed
P1 Margin	✓ Working	None (but interpret with care — see "When it fails")
P2 Input-grad	⚠ Partial	Side fix E: replace loss-grad with margin-grad to fix softmax-saturation bug
P3 Adversarial	✓ Working — gold standard	None
P4 Boundary thickness	✗ Broken on CIFAR	Side fix A: batched implementation
P5 Hessian top-k	✓ Working — gold standard	None
P6 Landscape	⚠ Visualization broken	Side fix C: auto-tune span; log-scale contours; eigenvector projection
P7 Linear interp	✓ Working	None
P8 Calibration	✓ Working	None (but it's not a brittleness probe)

Side fixes backlog

ID	What	Effort	Unblocks
A	Batched boundary-thickness probe (P4)	~30 min	P4 on CIFAR-scale models
C	Auto-tune landscape span from P5 (P6)	~15 min	Visual sharpness comparison in figures

ID	What	Effort	Bibliography
	Margin-gradient probe (replace P2)	~20 min	Onbooks: input-space sharpness without softmax saturation
Optional	ϵ -sharpness probe (Keskar's own metric)	~45 min	Direct apples-to-apples comparison with Keskar 2017's reported numbers

References

Full citations in `docs/REFERENCES.bib`. Key papers per probe:

- **P1:** No single source — margin distribution is standard ML diagnostic.
- **P2:** Implicit in adversarial-robustness literature.
- **P3:** Goodfellow 2015 (FGSM); Madry 2018 (PGD).
- **P4:** Yang et al. 2020 "Boundary thickness and robustness in learning models" (NeurIPS).
- **P5:** Pearlmutter 1994 "Fast Exact Multiplication by the Hessian"; Keskar 2017 for the connection to generalization.
- **P6:** Li et al. 2018 "Visualizing the Loss Landscape of Neural Nets" (NeurIPS).
- **P7:** Garipov et al. 2018 "Loss Surfaces, Mode Connectivity, and Fast Ensembling of DNNs" (NeurIPS).
- **P8:** Guo et al. 2017 "On Calibration of Modern Neural Networks" (ICML).